중고차 점검 사진 아카이브 기획서

https://gemini.google.com/app/cbdd0acfbdaea822

User prompt: 자동차 중고 판매처 헤이딜러의 차량보증을 담당하는 후배가 있어. 중고자동차를 보고 하자, 수리, 침수 등을 확인하고, 차량의 상태를 점검 및 보증하는 일을 하고 있는데. 챠량에 따라 고장이 잘 나는곳 등에 대한 사진 데이터를 많이 보관하고 있어 이를 체계적으로 잘 분류하고, 정리하고 싶어 제작계획서와 UI를 작성해줘.

Response: 후배분이 현업에서 쌓은 귀중한 차량 점검 데이터를 체계적인 자산으로 만들 수 있는 훌륭한 기획입니다. 파이썬(Python) 기반의 웹 프레임워크(Streamlit 등)와 이미지 처리 라이브러리(Pillow, OpenCV)를 활용하면 직관적이고 실용적인 사내용 데이터베이스를 빠르게 구축하실 수 있습니다.

요청하신 '중고차 점검 사진 데이터 분류 시스템(가칭: Car-chive)'의 제작 계획서와 UI 설계안을 정리해 드립니다.

1. 프로젝트 제작 계획서

프로젝트명: 헤이딜러 차량 점검 사진 아카이브 시스템 **목적:** 차종별/부위별 잦은 고장 및 하자(누유, 부식, 침수 등) 사진 데이터를 체계적으로 분류하여 점검 정확도를 높이고, 검수자 간의 노하우를 공유하는 지식 베이스 구축.

주요 기능 (Core Features):

- **이미지 업로드 및 태깅:** 사진 업로드 시 차종, 연식, 결함 부위, 결함 종류를 메타데이터로 저장.
- **다중 조건 검색:** 특정 차종의 고질적인 문제나 특정 하자(예: 침수 흔적)만 필터링하여 조회.
- **데이터 시각화 대시보드:** 차종별 주요 결함 발생 빈도 통계 제공.

추천 기술 스택:

- **Frontend / UI:** Streamlit (데이터 분석 및 대시보드 UI를 매우 빠르게 구현 가능)
- **Backend / DB:** Firebase 또는 SQLite (사진 메타데이터 및 태그 저장)
- **Image Processing:** Python (Pillow, OpenCV - 사진 리사이징 및 결함 부위 마킹용)

2. 사진 데이터 분류 체계 (Data Taxonomy)

사진을 업로드할 때 선택하게 될 핵심 카테고리입니다. 이 기준이 명확해야 나중에 검색이 쉬워집니다.

대분류 (부위)	중분류 (상세 위치)	소분류 (하자 및 상태)
파워트레인(엔진/미션)	엔진블록, 실린더헤드, 미션팬, 라디에이터	미세누유, 누유, 냉각수 혼입, 소음/진동 마모 흔적
하체 (Underbody)	프레임, 서스펜션, 조향장치, 배기구	부식(녹), 긁힘/파손, 누유, 고무부싱 파열
외관 및 골격	휠하우스, 쿼터패널, 도어, 필러	판금/도색 흔적, 스팟 용접 불량, 실링(접합부) 재작업
실내 및 전장	퓨즈박스, 안전벨트, 바닥 매트 하단	**침수(진흙/모래, 악취, 곰팡이)**, 배선 불량, 경고등

3. UI 화면 설계서 (Wireframe)

Streamlit과 같은 대시보드 형태의 레이아웃을 기준으로 한 4가지 주요 화면 설계입니다.

① 메인 대시보드 (Dashboard)

시스템 접속 시 가장 먼저 보이는 화면으로, 데이터의 흐름을 한눈에 파악합니다.

- **상단:** 전체 누적 사진 데이터 수, 이번 주 신규 업로드 건수.
- **중앙 (차트):** '최근 한 달간 가장 많이 등록된 하자 유형 TOP 5', '차종별 결함 발생 비율' (Pandas와 차트 라이브러리 연동).
- **하단:** 최근 업데이트된 점검 사진 썸네일 갤러리 (가로 스크롤).

② 사진 등록 화면 (Upload)

가장 빈번하게 사용될 화면이므로 입력 단계를 최소화해야 합니다.

- **좌측 영역 (Drag & Drop):** 점검 사진 파일 업로드 (다중 선택 지원).

- **우측 영역 (태그 입력 폼):**
  - 제조사/차종 선택 (Dropdown)
  - 연식 및 주행거리 (Input)
  - 점검 부위 및 하자 유형 (Selectbox - 위 데이터 분류 체계 연동)
  - 점검자 코멘트 (Text Area: "침수차량 판별을 위해 퓨즈박스 안쪽 모래알갱이 확인됨" 등)
- **하단:** [저장 및 분류] 버튼.

③ 데이터 검색 및 갤러리 (Search & Gallery)

특정 차종을 검수하러 가기 전, 주의해야 할 포인트를 찾아보는 화면입니다.

- **좌측 사이드바 (필터):** 차종 검색, 하자 유형(예: 침수), 부위(예: 하체) 멀티 셀렉트 필터.
- **메인 영역:** 필터링된 사진들이 바둑판(Grid) 형태로 정렬.
- **사진 썸네일 카드:** 사진과 함께 '차종, 연식, 결함유형'이 간략히 텍스트로 표시됨.

④ 상세 조회 화면 (Detail View)

썸네일을 클릭했을 때 나타나는 상세 화면입니다.

- **좌측:** 고해상도 원본 사진 크게 보기. (향후 OpenCV를 활용해 결함 부위에 빨간색 네모 박스를 치는 기능을 추가할 수 있습니다.)
- **우측:** 등록된 메타데이터 및 점검자 코멘트 전체 표시. 비슷한 결함을 가진 다른 차량의 사진 추천 링크.

User prompt: 사진과 태그 정보를 저장하기 위한 관계형 데이터베이스(또는 Firebase) 스키마를 상세하게 설계해 줘.

Response: 차량 점검 사진과 태그 데이터를 효율적으로 관리하고 검색하기 위한 데이터베이스 스키마 설계안입니다.

향후 데이터 통계 분석(Pandas 연동)이나 확장성을 고려하여 **관계형 데이터베이스(RDB)** 방식과, 파이썬 기반 웹 애플리케이션(Streamlit)에서 빠르고 유연하게 연동하기 좋은 **Firebase(NoSQL)** 방식 두 가지를 모두 준비했습니다.

옵션 1: 관계형 데이터베이스 (SQLite, PostgreSQL, MySQL)

사진 한 장에 여러 개의 하자 태그가 붙을 수 있으므로(예: 하나의 하체 사진에 '부식'과 '누유'가 동시에 존재), **다대다(N:M) 관계를 해소하는 매핑 테이블**을 두는 정규화 모델이 적합합니다.

1. `Vehicles` **(차량 기본 정보 테이블)**

동일한 차량에 대해 여러 부위의 사진이 업로드될 수 있으므로 차량 정보를 독립시킵니다.

컬럼명	데이터 타입	제약 조건	설명
`vehicle_id`	VARCHAR/UUID	**PK**	차량 고유 식별자 (또는 차량번호 해시값)
`manufacturer`	VARCHAR	NOT NULL	제조사 (예: 현대, 기아, BMW)
`model_name`	VARCHAR	NOT NULL	모델명 (예: 그랜저 IG)
`model_year`	INT		연식 (예: 2020)
`mileage`	INT		주행거리 (km 단위)

2. `Photos` **(사진 메타데이터 테이블)**

이미지 원본은 클라우드 스토리지(AWS S3, Firebase Storage 등)에 저장하고, DB에는 URL과 점검 정보만 저장합니다.

컬럼명	데이터 타입	제약 조건	설명
`photo_id`	VARCHAR/UUID	**PK**	사진 고유 식별자

컬럼명	데이터 타입	제약 조건	설명
vehicle_id	VARCHAR/UUID	**FK**	Vehicles 테이블 참조
image_url	VARCHAR	NOT NULL	스토리지에 저장된 사진 접근 URL
inspector_name	VARCHAR	NOT NULL	점검자 이름 또는 ID
comments	TEXT		점검자 특이사항 메모
created_at	TIMESTAMP	DEFAULT NOW	업로드 일시

3. Defect_Categories **(분류 체계 기준표 - 마스터 데이터)**

드롭다운 메뉴를 구성할 기준 데이터입니다. 하드코딩보다 DB에서 관리하는 것이 추후 항목 추가 시 유리합니다.

컬럼명	데이터 타입	제약 조건	설명
category_id	INT	**PK**, Auto Increment	카테고리 고유 식별자
main_part	VARCHAR	NOT NULL	대분류 (예: 파워트레인, 실내)
sub_part	VARCHAR	NOT NULL	중분류 (예: 엔진블록, 퓨즈박스)
defect_type	VARCHAR	NOT NULL	소분류/하자유형 (예: 미세누유, 침수)

4. Photo_Defects_Mapping **(사진-하자 태그 매핑 테이블)**

한 사진에 여러 하자가 있거나, 같은 유형의 하자가 여러 사진에 등장하는 것을 연결합니다.

컬럼명	데이터 타입	제약 조건	설명
mapping_id	INT	**PK**, Auto Increment	매핑 고유 식별자
photo_id	VARCHAR/UUID	**FK**	Photos 테이블 참조
category_id	INT	**FK**	Defect_Categories 테이블 참조

옵션 2: Firebase Firestore (NoSQL 문서형 데이터베이스)

초기 프로토타입 구축 속도를 높이고 스키마 변경에 유연하게 대응하려면 Firebase가 매우 효율적입니다. 조인(Join) 연산을 줄이기 위해 차량 정보와 결함 태그를 하나의 문서(Document) 안에 배열(Array)과 객체(Object) 형태로 비정규화하여 저장합니다.

컬렉션(Collection): inspection_photos *문서(Document) 예시:*

```
{
  "photo_id": "img_20261011_101530",
  "image_url": "gs://carchive-app.appspot.com/photos/img_20261011_101530.jpg",
  "vehicle_info": {
    "manufacturer": "현대",
    "model_name": "그랜저 IG",
    "model_year": 2019,
    "mileage": 85000
  },
  "defects": [
    {
      "main_part": "하체",
      "sub_part": "프레임",
      "defect_type": "부식"
    },
    {
      "main_part": "하체",
      "sub_part": "서스펜션",
      "defect_type": "고무부싱 파열"
    }
  ],
  "inspector_name": "홍길동",
  "comments": "조수석 프레임 하단부 심한 부식 확인됨. 염화칼슘 노출 의심.",
  "created_at": "2026-10-11T01:15:30Z"
}
```

Firebase 설계의 장점: Streamlit에서 `firebase-admin` 라이브러리를 통해 데이터를 불러오면, 곧바로 파이썬의 딕셔너리(Dictionary)나 리스트(List)로 변환되므로 Pandas 데이터프레임으로 만들어 통계 차트를 그리기가 매우 수월합니다.

User prompt: 이 데이터를 불러와서 차종이나 결함 부위로 다중 필터링을 할 수 있는 Streamlit 검색 UI 코드를 작성해 줘.

Response: Streamlit과 Pandas를 활용하여 다중 필터링 검색 UI를 구현한 파이썬 코드입니다.

실제 데이터베이스(Firebase)가 연결되기 전, UI의 동작을 바로 테스트해 볼 수 있도록 앞서 설계한 NoSQL 형태의 가상 데이터(Mock Data)를 내장하여 작성했습니다.

```
import streamlit as st
import pandas as pd

# 1. 페이지 기본 설정 (와이드 레이아웃)
st.set_page_config(page_title="Car-chive 검색 대시보드", layout="wide")

# 2. 가상 데이터 생성 (Firebase 문서 구조 기반)
@st.cache_data
def load_mock_data():
    data = [
        {
            "photo_id": "img_001",
            "image_url": "https://images.unsplash.com/photo-1580273916550-e323be2ae537?w=500&q=80",
            "manufacturer": "현대",
            "model_name": "그랜저 IG",
            "model_year": 2019,
            "defects": [{"main_part": "하체", "defect_type": "부식"}, {"main_part": "파워트레인", "defect_type": "미세누유"}],
            "inspector": "홍길동",
            "comments": "조수석 프레임 하단부 심한 부식 및 미션팬 누유"
        },
        {
            "photo_id": "img_002",
            "image_url": "https://images.unsplash.com/photo-1605270631317-068393526569?w=500&q=80",
            "manufacturer": "기아",
            "model_name": "K5 DL3",
            "model_year": 2021,
            "defects": [{"main_part": "실내", "defect_type": "침수(악취)"}],
            "inspector": "김철수",
            "comments": "안전벨트 끝단 진흙 자국 및 곰팡이 냄새 심함"
        },
        {
            "photo_id": "img_003",
            "image_url": "https://images.unsplash.com/photo-1549317661-bd32c8ce0db2?w=500&q=80",
            "manufacturer": "현대",
            "model_name": "쏘나타 DN8",
            "model_year": 2020,
            "defects": [{"main_part": "외관", "defect_type": "판금/도색"}],
            "inspector": "이영희",
            "comments": "운전석 쿼터패널 두께 이상, 스팟 용접 흔적 없음"
        },
        {
            "photo_id": "img_004",
            "image_url": "https://images.unsplash.com/photo-1619561005704-5183dbf9479b?w=500&q=80",
            "manufacturer": "BMW",
            "model_name": "520i",
            "model_year": 2018,
            "defects": [{"main_part": "파워트레인", "defect_type": "냉각수 혼입"}],
            "inspector": "홍길동",
            "comments": "엔진오일 캡 열었을 때 카푸치노 색상 유화 현상"
        }
    ]
    return pd.DataFrame(data)

df = load_mock_data()

# 3. 사이드바 다중 필터 UI 구성
st.sidebar.header("🔍 다중 조건 필터")

# 필터용 고유값 추출
manufacturers = df["manufacturer"].unique().tolist()
models = df["model_name"].unique().tolist()

# 결함 정보는 리스트 형태이므로 풀어(explode) 주어야 전체 목록을 얻을 수 있습니다.
all_defects_flat = [d for defects in df["defects"] for d in defects]
main_parts = list(set([d["main_part"] for d in all_defects_flat]))
defect_types = list(set([d["defect_type"] for d in all_defects_flat]))

# 멀티 셀렉트 박스
```

```
selected_mfg = st.sidebar.multiselect("제조사", manufacturers, default=[])
selected_model = st.sidebar.multiselect("차종", models, default=[])
selected_part = st.sidebar.multiselect("점검 부위", main_parts, default=[])
selected_defect = st.sidebar.multiselect("하자 유형", defect_types, default=[])

# 4. 데이터 필터링 로직
filtered_df = df.copy()

if selected_mfg:
    filtered_df = filtered_df[filtered_df["manufacturer"].isin(selected_mfg)]
if selected_model:
    filtered_df = filtered_df[filtered_df["model_name"].isin(selected_model)]
if selected_part:
    # 해당 차량의 여러 결함 중 하나라도 선택된 '점검 부위'를 포함하는지 확인
    filtered_df = filtered_df[filtered_df["defects"].apply(
        lambda defects: any(d["main_part"] in selected_part for d in defects)
    )]
if selected_defect:
    # 해당 차량의 여러 결함 중 하나라도 선택된 '하자 유형'을 포함하는지 확인
    filtered_df = filtered_df[filtered_df["defects"].apply(
        lambda defects: any(d["defect_type"] in selected_defect for d in defects)
    )]

# 5. 메인 대시보드 화면 렌더링
st.title("🚙 중고차 점검 사진 검색 (Car-chive)")
st.markdown("차종별, 부위별 고질적인 결함 및 점검 이력을 검색합니다.")

# 통계 지표
col1, col2, col3 = st.columns(3)
col1.metric(label="전체 등록 차량", value=f"{len(df)} 건")
col2.metric(label="검색된 차량", value=f"{len(filtered_df)} 건")
st.divider()

# 검색 결과 갤러리 뷰 (Grid 형태로 출력)
if filtered_df.empty:
    st.info("조건에 맞는 점검 데이터가 없습니다.")
else:
    # 3개씩 한 줄에 출력하기 위한 열 구성
    cols = st.columns(3)

    for idx, row in filtered_df.reset_index(drop=True).iterrows():
        col_idx = idx % 3
        with cols[col_idx]:
            # 카드 형태의 컨테이너
            with st.container(border=True):
                st.image(row["image_url"], use_column_width=True)
                st.subheader(f"{row['manufacturer']} {row['model_name']} ({row['model_year']})")

                # 하자 태그 출력
                tags = []
                for defect in row["defects"]:
                    tags.append(f"`{defect['main_part']}: {defect['defect_type']}`")
                st.markdown(" ".join(tags))

                st.caption(f"📝 점검자 코멘트: {row['comments']}")
```

코드 실행 방법 및 주요 로직 설명

1. **코드 실행:** 위 코드를 `app.py`로 저장한 후, 터미널에서 `streamlit run app.py`를 입력하면 로컬 브라우저에 대시보드가 열립니다.
2. **Pandas 최적화:** 차량 1대에 여러 결함 태그 리스트(JSON Array)가 존재합니다. `apply`와 `lambda` 함수를 사용하여 다중 조건(any)을 깔끔하게 필터링했습니다.
3. **그리드 레이아웃:** 검색 결과를 모바일이나 데스크톱 환경에서 보기 좋게 `st.columns(3)`와 모듈로 연산(`idx % 3`)을 결합하여 사진을 3열 바둑판(Grid) 형태로 나열했습니다.
